Finite Element Method

Julio A. Medina
University of San Carlos of Guatemala
School of Physical Sciences and Mathematics
Master's Program in Physics
julioantonio.medina@gmail.com

Abstract

This report provides a theoretical introduction to the Finite Element Method (FEM) for solving partial differential equations subject to boundary conditions. It also presents relatively concise implementations for elliptic, parabolic, and hyperbolic partial differential equations. The discussion includes the tessellation, or domain-discretization, process and compares computational implementations in `Wolfram Mathematica` and `Python`.

1 Overview of the Finite Element Method

There are several theoretical approaches to the finite element method. The introduction developed in this document follows a functional approach, in which the objective is to minimize a functional constructed from the specific problem under consideration. This is the approach presented in [**?**], analogous to the Ritz method.
A brief account of the weak formulation is also included. Starting from the partial differential equation, an integral statement is derived that must hold for suitable test functions. This formalism is required to solve the problems with the `Python` library `FEniCS`. Both approaches are rooted in the calculus of variations.

2 Introduction to the Finite Element Method

This method for approximating solutions of partial differential equations was originally developed for civil-engineering applications. It is now used throughout applied mathematics and in many areas of physics. FEM is employed extensively in the modeling of complex systems, the aerodynamic design of components, space-exploration technology, and advanced engineering applications such as Formula One racing.

One advantage of the finite element method over the finite difference method is the relative ease with which FEM handles boundary conditions. Many physical problems have boundary conditions involving derivatives and irregular boundaries without apparent symmetries. Such conditions can be difficult to treat with finite differences because every derivative boundary condition must be approximated by a difference quotient at grid points, while irregular boundaries make the construction of a suitable grid nontrivial. In FEM, boundary conditions can instead enter through the variational formulation, so the overall construction is

better suited to irregular geometries and mixed boundary conditions.

The partial differential equation considered is

$$\frac{\partial}{\partial x}\left(p(x,y)\frac{\partial u}{\partial x}\right)+\frac{\partial}{\partial y}\left(q(x,y)\frac{\partial u}{\partial y}\right)+r(x,y)u(x,y)=f(x,y) \tag{1}$$

for $(x,y)\in\mathcal{D}$, where $\mathcal{D}$ is a planar region with boundary $\mathcal{S}$.

Boundary conditions of the form

$$u(x,y)=g(x,y) \tag{2}$$

are imposed on a portion $\mathcal{S}_1$ of the boundary. On the remaining portion, $\mathcal{S}_2$, the solution is required to satisfy

$$p(x,y)\frac{\partial u}{\partial x}(x,y)\cos\theta_1+q(x,y)\frac{\partial u}{\partial y}(x,y)\cos\theta_2+g_1(x,y)u(x,y)=g_2(x,y), \tag{3}$$

where θ_1 and θ_2 are the direction angles of the outward normal at the point (x,y); see Figure **??**. Physical problems in areas such as solid mechanics and

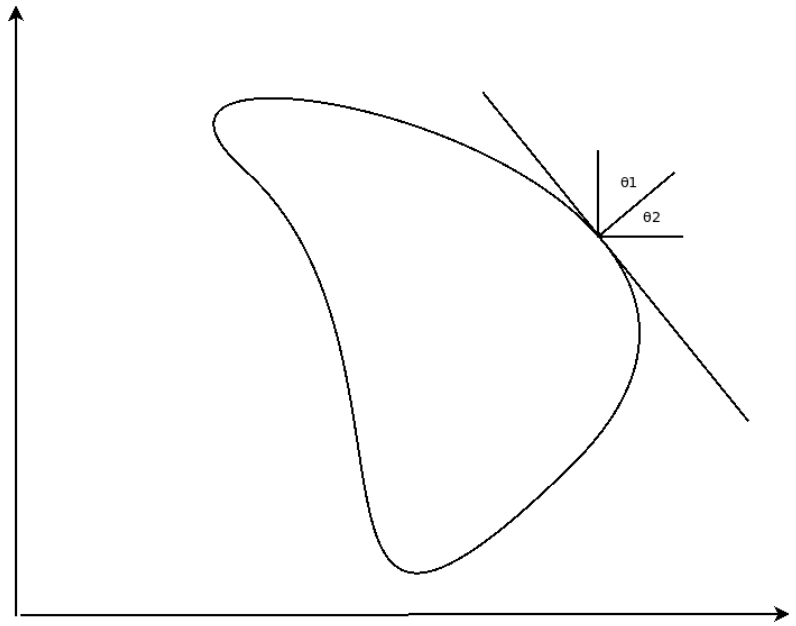

Figure 1: Direction angles θ_1 and θ_2

elasticity often involve partial differential equations similar to **??**. Their solutions typically minimize a functional involving definite integrals over a class of functions determined by the problem.

Assume that p, q, r, and f are continuous on $\mathcal{D}\cup\mathcal{S}$; that p and q have continuous first partial derivatives; and that g_1 and g_2 are continuous on $\mathcal{S}_2$.

In addition, assume that $p(x,y) > 0$, $q(x,y) > 0$, $r(x,y) \leq 0$, and $g_1(x,y) > 0$. Then the solution of **??** is the unique minimizer of the functional $I[w]$.

$$I[w] = \int\int_{\mathcal{D}} \left\{ \frac{1}{2}\Big[p(x,y)\Big(\frac{\partial w}{\partial x}\Big)^2 + q(x,y)\Big(\frac{\partial w}{\partial y}\Big)^2 - r(x,y)w^2\Big] + f(x,y)w \right\} dxdy + \int_{\mathcal{S}_2} \left\{ -g_2(x,y)w + \frac{1}{2}g_1(x,y)w^2 \right\} dS \tag{4}$$

over all twice-differentiable continuous functions w that satisfy **??** on $\mathcal{S}_1$. The finite element method approximates this solution by minimizing the functional I in **??** over a smaller finite-dimensional class of functions, similarly to the Rayleigh–Ritz approach; see [**?**].

2.1 Defining the Finite Elements

The first step is to define the finite elements used to construct the approximation. The region is divided into a finite number of regularly shaped sections or elements, such as triangles or rectangles, that tessellate and discretize the entire domain.

The approximation space is commonly formed by piecewise polynomials of fixed degree in x and y. The pieces are joined so that the resulting function is continuous and has the required weak derivatives over the whole region. Linear polynomials in x and y of the form

$$\phi(x,y) = a + bx + cy \tag{5}$$

are commonly used with triangular elements, whereas bilinear polynomials in x and y of the form

$$\phi(x,y) = a + bx + cy + dxy \tag{6}$$

are associated with rectangular elements.

Assume that the region $\mathcal{D}$ has been subdivided into triangular elements. Let D denote the collection of triangles, whose vertices are called nodes. The method seeks an approximation of the form

$$\phi(x,y) = \sum_{i=1}^{m} \gamma_i \phi_i(x,y), \tag{7}$$

where $\phi_1, \phi_2, \ldots, \phi_m$ are linearly independent piecewise-linear polynomials and $\gamma_1, \gamma_2, \ldots, \gamma_m$ are constants. Some of these constants, for example $\gamma_{n+1}, \gamma_{n+2}, \ldots, \gamma_m$, are used to enforce the boundary condition

$$\phi(x,y) = g(x,y) \tag{8}$$

on $\mathcal{S}_1$. The remaining constants $\gamma_1, \gamma_2, \ldots, \gamma_n$ are chosen to minimize $I[\sum_{i=1}^{m} \gamma_i \phi_i]$.

Substituting the approximation **??** for w in **??** gives

$$\begin{aligned} I[\phi] &= I\Big[\sum_{i=1}^{m}\gamma_i\phi_i\Big] \\ &= \int\int_{\mathcal{D}}\Bigg(\frac{1}{2}\Bigg\{p(x,y)\Big[\sum_{i=1}^{m}\gamma_i\frac{\partial\phi_i}{\partial x}(x,y)\Big]^2 + q(x,y)\Big[\sum_{i=1}^{m}\gamma_i\frac{\partial\phi_i}{\partial y}(x,y)\Big]^2 \\ &- r(x,y)\Big[\sum_{i=1}^{m}\gamma_i\phi_i(x,y)\Big]^2\Bigg\} + f(x,y)\sum_{i=1}^{m}\gamma_i\phi_i(x,y)\Bigg)dy\,dx \\ &+ \int_{\mathcal{S}_2}\Bigg\{-g_2(x,y)\sum_{i=1}^{m}\gamma_i\phi_i(x,y) + \frac{1}{2}g_1(x,y)\Big[\sum_{i=1}^{m}\gamma_i\phi_i(x,y)\Big]^2\Bigg\}dS \end{aligned} \tag{9}$$

As noted above, $\gamma_1, \gamma_2, \ldots, \gamma_n$ are selected to minimize the functional. Treating I as a function $I[\gamma_1, \gamma_2, \ldots, \gamma_n]$, a necessary condition for a minimum is

$$\frac{dI}{d\gamma_j} = 0,\ \forall\, j = 1, 2, \ldots, n. \tag{10}$$

Differentiating **??** with respect to γ_j gives

$$\begin{aligned} \frac{\partial I}{\partial\gamma_j} &= \int\int_{\mathcal{D}}\Bigg\{p(x,y)\sum_{i=1}^{m}\gamma_i\frac{\partial\phi_i}{\partial x}(x,y)\frac{\partial\phi_j}{\partial x}(x,y) \\ &+ q(x,y)\sum_{i=1}^{m}\gamma_i\frac{\partial\phi_i}{\partial y}(x,y)\frac{\partial\phi_j}{\partial y}(x,y) \\ &- r(x,y)\sum_{i=1}^{m}\gamma_i\phi_i(x,y)\phi_j(x,y) + f(x,y)\phi_j(x,y)\Bigg\}dx\,dy \\ &+ \int_{\mathcal{S}_2}\Bigg\{-g_2(x,y)\phi_j(x,y) + g_1(x,y)\sum_{i=1}^{m}\gamma_i\phi_i(x,y)\phi_j(x,y)\Bigg\}dS, \end{aligned} \tag{11}$$

and therefore the stationarity condition becomes

$$\begin{aligned} 0 = \sum_{i=1}^{m}\Bigg[&\int\int_{\mathcal{D}}\Bigg\{p(x,y)\frac{\partial\phi_i}{\partial x}(x,y)\frac{\partial\phi_j}{\partial x}(x,y) + q(x,y)\frac{\partial\phi_i}{\partial y}(x,y)\frac{\partial\phi_j}{\partial y}(x,y) \\ &- r(x,y)\phi_i(x,y)\phi_j(x,y)\Bigg\}dx\,dy \\ &+ \int_{\mathcal{S}_2} g_1(x,y)\phi_i(x,y)\phi_j(x,y)dS\Bigg]\gamma_i \\ &+ \int\int_{\mathcal{D}} f(x,y)\phi_j(x,y)dx\,dy - \int_{\mathcal{S}_2} g_2(x,y)\phi_j(x,y)dS. \end{aligned} \tag{12}$$

for each $j = 1, 2, \ldots, n$. This system of equations can be written in matrix form as

$$\mathbf{A}\vec{c} = \vec{b}, \tag{13}$$

where $\vec{c} = (\gamma_1, \ldots, \gamma_n)^t$. The matrix $\mathbf{A} = (\alpha_{ij})$ and vector $\vec{b} = (\beta_1, \ldots, \beta_n)^t$ are defined below.

$$\begin{aligned}\alpha_{ij} = \int\int_{\mathcal{D}} \Bigg[& p(x,y)\frac{\partial \phi_i}{\partial x}(x,y)\frac{\partial \phi_j}{\partial x}(x,y) + q(x,y)\frac{\partial \phi_i}{\partial y}(x,y)\frac{\partial \phi_j}{\partial y}(x,y) \\ & -r(x,y)\phi_i(x,y)\phi_j(x,y)\Bigg] dx\, dy + \int_{\mathcal{S}_2} g_1(x,y)\phi_i(x,y)\phi_j(x,y) dS,\end{aligned} \tag{14}$$

for each $i = 1, 2, \ldots, n$ and $j = 1, 2, \ldots, m$, and

$$\beta_i = -\int\int_{\mathcal{D}} f(x,y)\phi_i(x,y) dx\, dy + \int_{\mathcal{S}} g_2(x,y)\phi_i(x,y) dS - \sum_{k=n+1}^{m} \alpha_{ik}\gamma_k. \tag{15}$$

for each $i = 1, \ldots, n$.

The choice of basis functions is important because an appropriate selection can often make $\mathbf{A}$ positive definite and give it a banded-matrix structure. For the second-order problem **??**, the domain $\mathcal{D}$ is assumed to be polygonal, so that $\mathcal{D} = D$, and $\mathcal{S}$ is a set of contiguous straight-line segments.

2.2 Triangulating the Region

To begin the procedure described above, the region D must be subdivided into triangles $T_1, T_2, \ldots, T_M$. Triangle i has three vertices, or nodes, denoted by

$$V_j^{(i)} = \left(x_j^{(i)}, y_j^{(i)}\right), \text{ for } j = 1, 2, 3. \tag{16}$$

For simplicity, write $V_j^{(i)} = \left(x_j^{(i)}, y_j^{(i)}\right)$ while working with a fixed triangle T_i. Associated with each vertex V_j is a linear polynomial

$$N_j^{(i)}(x,y) \equiv N_j(x,y) = a_j + b_j x + c_j y, \text{ where } N_j^{(i)}(x_k, y_k) = \begin{cases} 1, & j = k \\ 0, & j \neq k \end{cases} \tag{17}$$

This construction produces a linear system of the form

$$\begin{bmatrix} 1 & x_1 & y_1 \\ 1 & x_2 & y_2 \\ 1 & x_3 & y_3 \end{bmatrix} \begin{bmatrix} a_j \\ b_j \\ c_j \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}. \tag{18}$$

In **??**, $j = 2$, so the entry equal to 1 occurs in row 2. In general, the entry equal to 1 occurs in row j.

Suppose that the nodes $E_1, \ldots, E_n$ in $D \cap \mathcal{S}$ have been labeled. Each node E_k is associated with a function ϕ_k that is linear on every triangle, equals 1 at E_k, and equals 0 at every other node. This construction makes ϕ_k identical to $N_j^{(i)}$ on triangle T_i whenever E_k is the vertex denoted by $V_j^{(i)}$.

3 Weak Formulation for FEM

The discussion begins with the original problem, also known as the strong formulation. Consider the elliptic partial differential equation given by the Poisson equation

$$\nabla^2 T = f(x) \tag{19}$$

with the Dirichlet condition

$$T = u(x), \quad x \in \Gamma_D \tag{20}$$

and the Neumann condition

$$\nabla T \cdot \mathbf{n} = g(x), \quad x \in \Gamma_N \tag{21}$$

where $\Omega \subset \mathbb{R}^n$ is the solution domain with boundary $\partial\Omega$, Γ_D is the portion of the boundary on which the Dirichlet condition is imposed, and Γ_N is the portion with a Neumann condition. The function $T(x)$ is the unknown to be found or approximated, while $f(x)$, $u(x)$, and $g(x)$ are known functions.
The weak form of **??** is obtained by requiring

$$\int_\Omega (\nabla^2 T - f) \cdot s \, d\Omega = 0 \tag{22}$$

where s is a test function. Integration by parts gives

$$\begin{aligned} 0 = \int_\Omega (\nabla^2 T - f) \cdot s \, d\Omega = \int_\Omega \nabla \cdot \big(\nabla T\big) \cdot s \, d\Omega - \int_\Omega f \cdot s d\Omega = \\ = -\int_\Omega \nabla T \cdot \nabla s \, d\Omega + \int_\Omega \nabla \cdot \big(\nabla T \cdot s\big) \, d\Omega - \int_\Omega f \cdot s \, d\Omega \end{aligned} \tag{23}$$

Applying the divergence theorem yields

$$\int_\Omega \nabla T \cdot \nabla s \, d\Omega = \int\limits_{\Gamma_D \cup \Gamma_N} s \, (\nabla T \cdot \mathbf{n}) \, d\Gamma - \int_\Omega f \, s \, d\Omega \tag{24}$$

The boundary integral can be separated into a contribution from the Dirichlet portion and another from the Neumann portion:

$$\int_\Omega \nabla T \cdot \nabla s \, d\Omega = \int\limits_{\Gamma_D} s \, (\nabla T \cdot \mathbf{n}) \, d\Gamma + \int\limits_{\Gamma_N} s \, (\nabla T \cdot \mathbf{n}) \, d\Gamma - \int_\Omega f \, s \, d\Omega \tag{25}$$

Equation **??** is the initial weak form of the Poisson equation, but it cannot be applied directly without first accounting for the boundary conditions.

3.1 Dirichlet Boundary Conditions

On the portion of the boundary where Dirichlet conditions apply, there are two constraints: the boundary condition $T(x) = u(x)$ and the boundary integral over Γ_D in **??**. To eliminate that boundary-integral term while enforcing the prescribed value, take $T \in V(\Omega)$ and the test function $s \in V_0(\Omega)$, where

$$\begin{aligned} V(\Omega) &= \{v(x) \in H^1(\Omega)\}, \\ V_0(\Omega) &= \{v(x) \in H^1(\Omega);\ v(x) = 0,\ x \in \Gamma_D\}. \end{aligned}$$

In Sobolev-space terms, the unknown T and the test function s belong to $H^1(\Omega)$, so they and their first weak derivatives are square integrable. In addition, the test function vanishes on Γ_D.
With this requirement, the boundary term over Γ_D vanishes. The weak form of the Poisson equation for $T \in V(\Omega)$ and $s \in V_0(\Omega)$ is

$$\begin{aligned} \int_\Omega \nabla T \cdot \nabla s \, d\Omega = \int_{\Gamma_N} s \, (\nabla T \cdot \mathbf{n}) \, d\Gamma - \int_\Omega f \, s \, d\Omega \\ T(x) = u(x), \;\; x \in \Gamma_D. \end{aligned} \tag{26}$$

For this reason, Dirichlet conditions are called *essential boundary conditions* in FEM terminology: they are not incorporated automatically by the weak formulation and must be imposed explicitly.

3.2 Neumann Boundary Conditions

Neumann boundary conditions prescribe the flux $g(x) = \nabla T \cdot \mathbf{n}$. The boundary integral over Γ_N in **??** contains exactly this flux, so $g(x)$ can be substituted directly into the integral:

$$\int_\Omega \nabla T \cdot \nabla s \, d\Omega = \int_{\Gamma_N} g \cdot s \, d\Gamma - \int_\Omega f \cdot s \, d\Omega, \tag{27}$$

where the test function s also belongs to V_0.
Neumann conditions are therefore called *natural boundary conditions*, because they appear naturally in the weak formulation.

3.3 Weak Formulation of the Poisson Equation

Combining the previous results gives the weak formulation of the Poisson equation **??**: for every test function $s \in V_0(\Omega)$, find $T \in V(\Omega)$ such that

$$\begin{aligned} \int_\Omega \nabla T \cdot \nabla s \, d\Omega = \int_{\Gamma_N} g \cdot s \, d\Gamma - \int_\Omega f \cdot s \, d\Omega, \\ T(x) = u(x), \;\; x \in \Gamma_D. \end{aligned} \tag{28}$$

4 Implementation Analysis

This section analyzes finite element implementations in **Wolfram Mathematica 12** and **Python**, using the **FEniCS** library in the latter case; see [**?**]. Three classes of partial differential equations are considered:

- Elliptic equation
- Hyperbolic equation
- Parabolic equation

4.1 Elliptic Equation

The elliptic partial differential equation to be solved is

$$\begin{aligned} \nabla^2 u(x,y) &= xe^y \\ \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} &= xe^y, \;\; 0 < x < 2, \;\; 0 < y < 1 \end{aligned} \tag{29}$$

with boundary conditions

$$\begin{aligned} u(0,y) = 0, \;\; u(2,y) = 2e^y, \;\; 0 \le y \le 1, \\ u(x,0) = x, \;\; u(x,1) = ex, \;\; 0 \le x \le 2. \end{aligned} \tag{30}$$

This elliptic PDE is the well-known Poisson equation, here considered in two spatial dimensions.

4.1.1 Implementation in Mathematica

Solving **??** with the `Needs["NDSolve‘FEM‘"]` package in `Wolfram Mathematica 12` gives a numerical solution that can be visualized as a surface embedded in three-dimensional space.

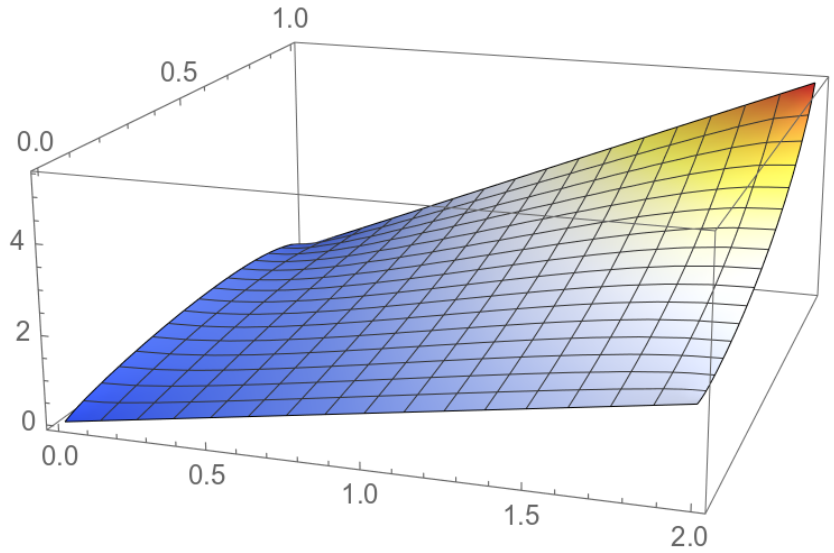

Figure 2: Numerical solution of the elliptic PDE

To demonstrate the geometric capabilities of Mathematicaś finite element tools, consider solving the elliptic equation on the rectangular region defined above while excluding the points inside the circle described by

$$(x - 0.2)^2 + (y - 0.2)^2 \le 0.3^2 \tag{31}$$

This produces the following planar region:

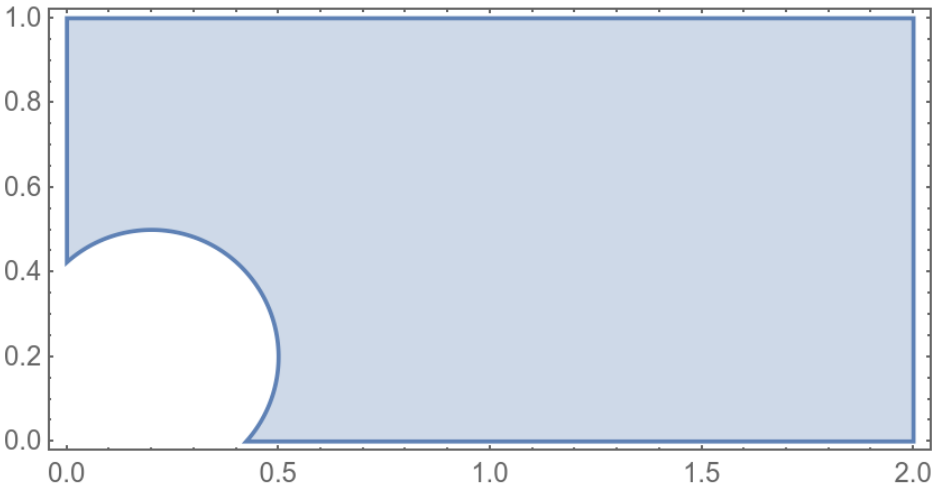

Figure 3: Region excluding the points described by **??**

In addition to making the region more complex, impose Dirichlet conditions of the form

$$u(x,y) = \begin{cases} 0, & x = 2, \quad 0.8 \le y \le 1.0 \\ 10, & (x-0.2)^2 + (y-0.2)^2 \le 0.3^2 \end{cases} \tag{32}$$

Solving the problem gives the following contour lines:

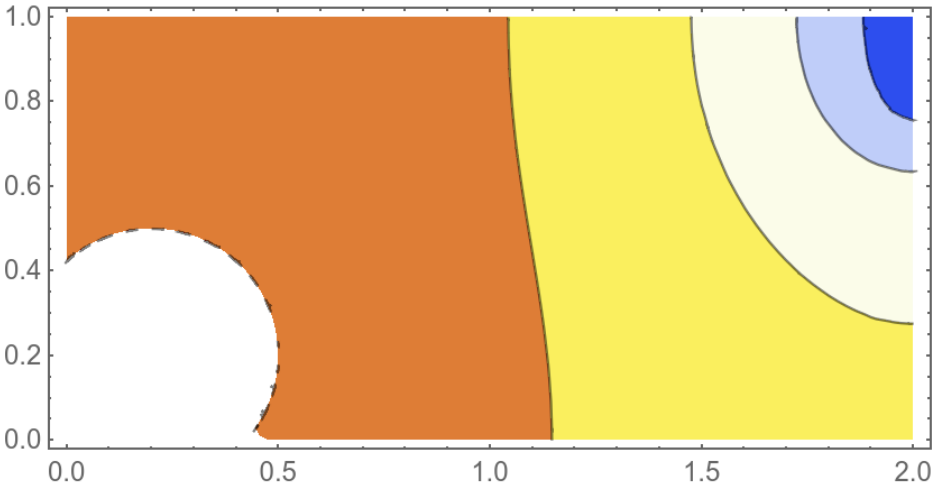

Figure 4: Contour lines

The corresponding solution in three-dimensional space is shown below. The tessellation of the domain generated by the finite element package can also be visualized:

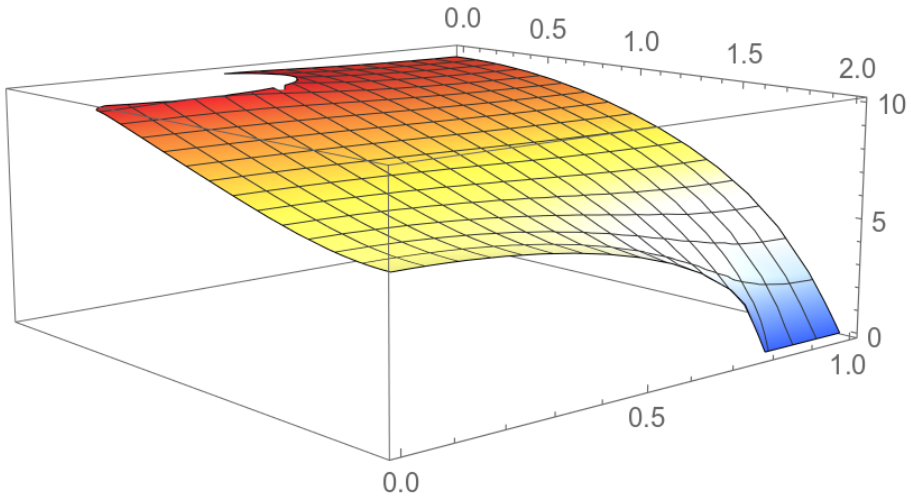

Figure 5: Numerical solution with Dirichlet boundary conditions

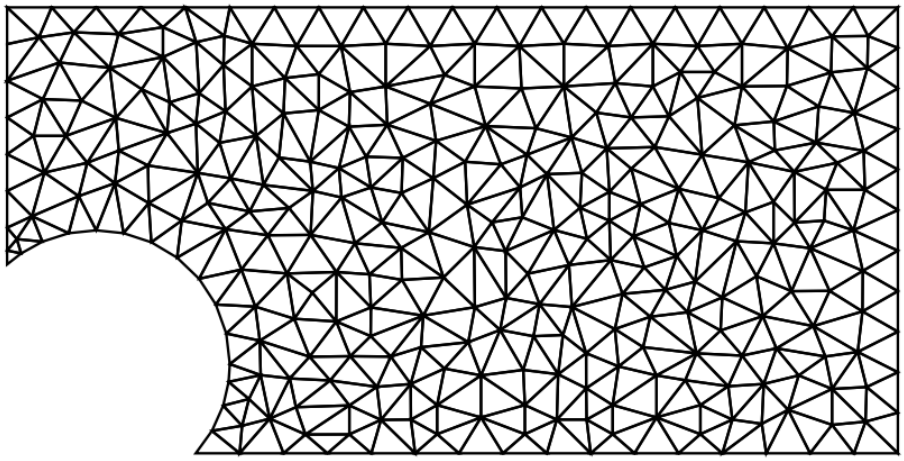

Figure 6: Tessellation of the domain

4.1.2 Implementation in FEniCS

To use `FEniCS`, the PDE must be written in weak form. As discussed above, the terms in **??** can be used to express the elliptic, or Poisson, problem in code. Applying this procedure to **??** gives the following approximation:

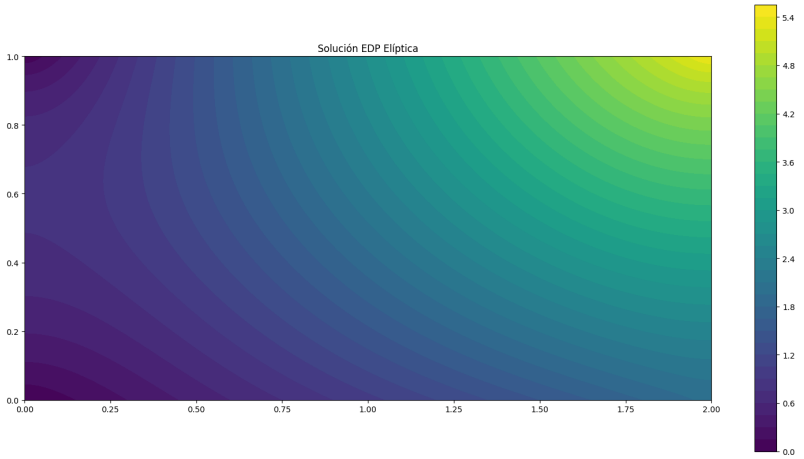

Figure 7: Numerical solution of the Poisson equation

4.2 Parabolic Equation

For the parabolic equation, consider the problem

$$\frac{\partial u}{\partial t}(x,t) - \frac{\partial^2 u}{\partial x^2}(x,t) = 0, \quad 0 < x < 1, \quad t \geq 0 \tag{33}$$

with boundary conditions

$$u(0,t) = u(1,t) = 0, \quad t > 0 \tag{34}$$

and initial condition

$$u(x,0) = \sin(\pi x), \quad 0 \leq x \leq 1 \tag{35}$$

4.2.1 Implementation in Mathematica

Using `Needs["NDSolve‘FEM‘"]` and `NDSolveValue`, the PDE and its initial and boundary conditions can be specified directly. The numerical approximation is shown below.

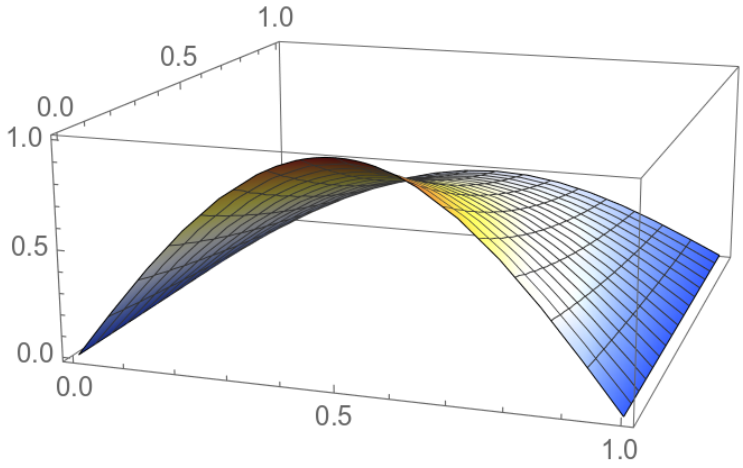

Figure 8: Numerical approximation of the parabolic PDE

It is also useful to visualize the contour lines of the approximate numerical solution.

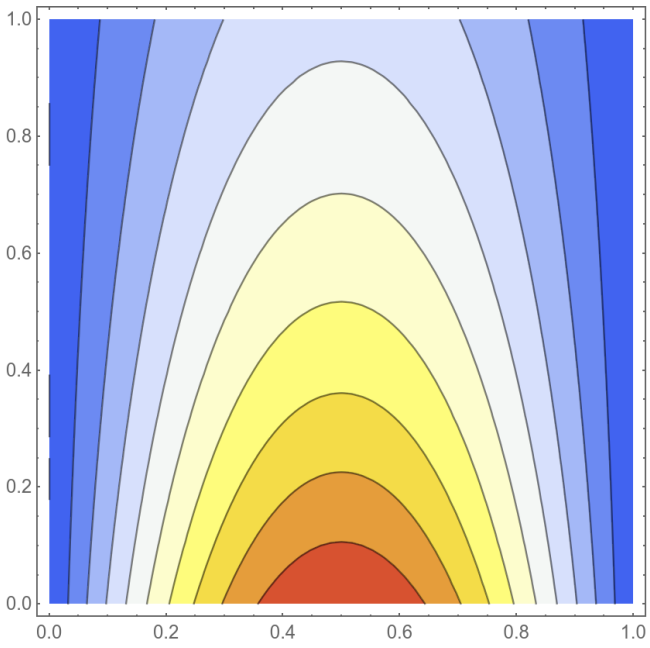

Figure 9: Contour lines of the numerical approximation to the parabolic PDE

4.2.2 Implementation in FEniCS

To solve the equation numerically, the weak form of **??** must be constructed. The corresponding `Python` code is available in this repository; see `https://`

`github.com/Julio-Medina/Finite_Element_Method`.
The result obtained with FEniCS is shown below.

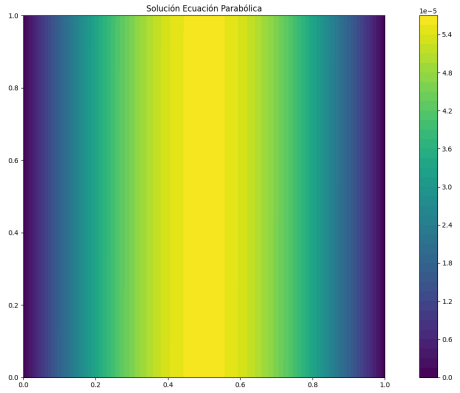

Figure 10: Approximation to the solution of the parabolic PDE

The mesh produced by `FEniCS` can be plotted over the solution to show the tessellation of the computational domain.

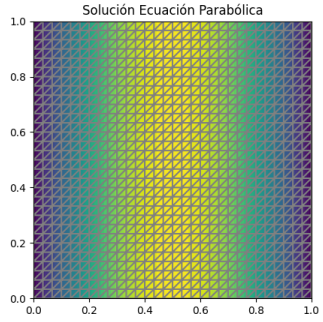

Figure 11: Tessellation used for the parabolic PDE

4.3 Hyperbolic Equation

The hyperbolic equation considered here is the wave equation. The problem is

$$\frac{\partial^2 u}{\partial t^2}(x,y) - 4\frac{\partial^2 u}{\partial x^2}(x,t) = 0, \;\; 0 < x < 1, \;\; t > 0 \tag{36}$$

with boundary conditions

$$u(0,t) = u(1,t) = 0, \;\; \text{for} \;\; t > 0 \tag{37}$$

and initial conditions

$$u(x,0) = \sin(\pi x), \;\; \text{y} \;\; \frac{\partial u}{\partial t}(x,0) = 0, \;\; 0 \leq x \leq 1 \tag{38}$$

4.3.1 Implementation in Mathematica

The result of solving **??** with its boundary and initial conditions in Mathematica is shown below.

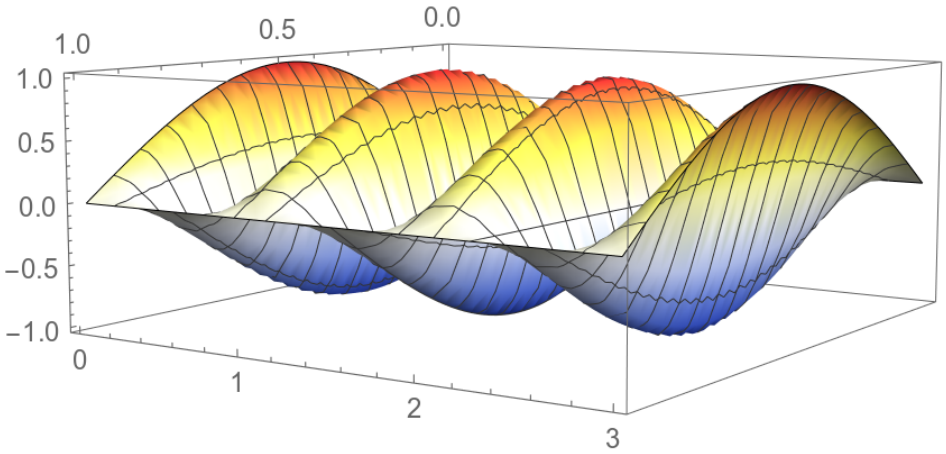

Figure 12: Numerical solution of the wave equation

The corresponding contour lines are shown in the following figure.

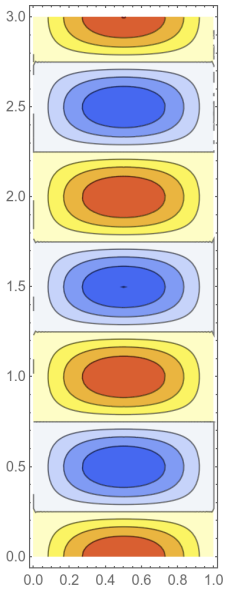

Figure 13: Contour lines of the numerical wave-equation solution

4.3.2 Implementation in FEniCS

Using the weak formulation of **??**, the problem can be treated with `FEniCS`. The result is shown below.

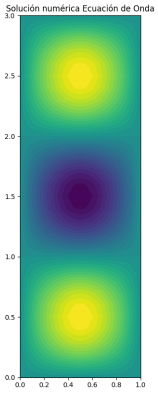

Figure 14: Numerical solution of the wave equation

5 Comparison of the Implementations

To compare the implementations for the three canonical PDE classes, the average RAM consumption of each package was recorded.

Partial Differential Equation	`Wolfram Mathematica`	`Python/FEniCS`
Elliptic	188 MB	266 MB
Parabolic	178 MB	148 MB
Hyperbolic	178 MB	152 MB

5.1 Spatial Discretization

Discretizing the domain is a fundamental step in the finite element method, and many theoretical approaches are available. As discussed previously, the choice of elementary geometric cell used for the tessellation directly affects the construction and quality of the method.

Some of the mesh-generation ideas associated with the `NDSolve‘FEM‘` framework in `Wolfram Mathematica` include the following:

1. **Delaunay triangulation**: This is a common algorithm for generating two-dimensional meshes. A triangulation of a set of points is Delaunay if no point lies inside the circumcircle of any triangle. Delaunay triangulations tend to maximize the minimum triangle angle, producing comparatively well-shaped elements.

2. **Voronoi diagram**: This is the geometric dual of a Delaunay triangulation. Given a set of points, each point is assigned a polygon containing

the locations closer to that point than to any other. These polygons form a Voronoi diagram and can support mesh-refinement strategies.

3. **Octree method (quadtree in 2D)**: The domain is divided recursively into eight equal parts in three dimensions, or four equal parts in two dimensions, until the desired resolution is reached. Specialized transition elements can connect regions with different levels of refinement.

4. **Advancing-front method**: The procedure starts from a boundary mesh and progressively adds elements toward the interior. A front is maintained between meshed and unmeshed regions, and new nodes are introduced so that the front advances.

5. **Ruppertś algorithm**: This is a Delaunay-refinement algorithm that also enforces a minimum-angle criterion, producing high-quality triangulations.

`FEniCS` uses several related techniques and theoretical frameworks. As an open-source project, it also offers the following strengths:

1. **Mesh generation**: FEniCS workflows can use libraries such as `mshr` and `CGAL` to create and manipulate complex meshes.

2. **Automated finite element assembly**: A principal strength of `FEniCS` is its automated assembly of finite element systems. Variational forms, which provide the mathematical foundation of finite element discretization, can be expressed in notation close to the underlying mathematics directly in `Python` or `C++`.

3. **Problem solution**: `FEniCS` uses `PETSc` to solve linear and nonlinear systems, providing powerful and scalable numerical solvers.

6 Statistical Analysis of Numerical Approximation Results

Statistical analysis can be applied to the output of a numerical approximation to better understand the characteristics and performance of the method, validate the results, and draw inferences about the process that generated the data. Common techniques include:

1. **Descriptive statistics**: The mean, median, mode, standard deviation, variance, range, minimum, maximum, quartiles, and related measures summarize the central tendency, dispersion, and distribution of the data.

2. **Error analysis**: Residuals, defined as differences between observed and predicted values, can be calculated and their distribution examined. Metrics such as mean absolute error and mean squared error may be used, and residual plots can reveal systematic patterns.

3. **Confidence intervals**: Confidence intervals around estimated parameters or predictions can be constructed to quantify uncertainty.

4. **Hypothesis testing**: When a specific hypothesis is of interest, such as whether one approximation method outperforms another, an appropriate statistical test can be applied. For example, a t-test can compare the means of two groups.

5. **Correlation analysis**: When several variables are present, their relationships can be examined with correlation coefficients or scatter plots.

6. **Regression analysis**: When the results depend on multiple input variables, regression can help explain their collective influence and may also support prediction.

7. **Analysis of variance (ANOVA)**: When comparing several approximation methods or several configurations of one method, ANOVA can assess whether the differences among groups are statistically significant.

8. **Bootstrap or cross-validation**: Resampling or data-splitting techniques can be used to estimate the accuracy and stability of the approximation method.

9. **Monte Carlo analysis**: If the numerical approximation involves randomness, as in Monte Carlo integration, the distribution of results over repeated runs can be analyzed through its mean, variance, and other statistical properties.

7 Conclusions

The finite element method has become a fundamental tool in mathematical modeling, physical simulation, and engineering analysis. Partial differential equations subject to Dirichlet or Neumann boundary conditions are often difficult to solve analytically. A notable graduate-level example in physics is the study of electromagnetic boundary-value problems discussed by Jackson [**?**].

Computational packages are therefore used throughout science, since most realistic problems do not admit closed-form analytical solutions. FEM is one of the most widely used numerical approaches in practice, particularly because of its ability to handle complex geometries and boundary conditions.

The results obtained for the three canonical PDE classes show that `Wolfram Mathematica` and `FEniCS` both provide robust and relatively accessible FEM implementations, each with advantages and disadvantages. Mathematica allows the problem to be formulated in a syntax close to ordinary mathematical notation: the PDE can be entered almost directly. FEniCS requires the user to derive and define the weak formulation explicitly. Mathematica is proprietary software, whereas FEniCS is open source, making FEniCS more accessible and configurable at the cost of requiring more detailed programming and numerical knowledge. For the examples considered here, the differences in RAM consumption are not substantial, although both Python and Mathematica are known to use significant memory.
With respect to execution time, Mathematica solved these examples almost immediately, whereas the Python implementation could require up to several

minutes depending on the selected parameters. A future comparison with a `C++` implementation would provide a more informative performance benchmark.

References

[1] Richard L. Burden, J. Douglas Faires *Numerical Analysis*, (Ninth Edition). Brooks/Cole, Cengage Learning. 978-0-538-73351-9

[2] Cimrman, R., Lukeš, V., Rohan, E., 2019. *Multiscale finite element calculations in Python using SfePy.* Advances in Computational Mathematics 45, 1897-1921. https://doi.org/10.1007/s10444-019-09666-0

[3] M. S. Alnaes, J. Blechta, J. Hake, A. Johansson, B. Kehlet, A. Logg, C. Richardson, J. Ring, M. E. Rognes and G. N. Wells. *The FEniCS Project Version 1.5, Archive of Numerical Software 3 (2015).* [doi.org/10.11588/ans.2015.100.20553]

[4] A. Logg, K.-A. Mardal, G. N. Wells. *Automated Solution of Differential Equations by the Finite Element Method*, Springer(2012). [doi.org/10.1007/978-3-642-23099-8]

[5] John David Jackson. *Classical Electrodynamics.* Wiley, 1999.